

## LAB 12. GỬI EMAIL CHÀO MỪNG QUA HỆ THỐNG QUEUE (TÁC VỤ CHẠY NỀN)

Ở Lab 10, bạn đã tích hợp thành công hệ thống Đăng ký/Đăng nhập (Laravel Breeze). Tuy nhiên, hiện tại người dùng đăng ký xong chưa nhận được thông báo qua email.

### MỤC TIÊU LAB:

Bạn sẽ xây dựng tính năng tự động gửi email chào mừng ngay khi người dùng đăng ký tài khoản thành công. Để đảm bảo người dùng không phải chờ đợi quá trình gửi thư (thường mất 3-5 giây), bạn sẽ cấu hình ép bức thư này vào **Hàng đợi (Queue)** và để hệ thống xử lý ngầm.

### HƯỚNG DẪN THỰC HÀNH

#### Bước 1. Cấu hình hệ thống Queue (Database Driver):

Để Laravel có chỗ lưu trữ các tác vụ gửi email đang xếp hàng chờ, chúng ta sẽ sử dụng Database.

1. Mở file .env tại thư mục gốc, tìm biến QUEUE\_CONNECTION và đổi giá trị thành database:

```
QUEUE_CONNECTION=database
```

2. Mở Terminal và chạy các lệnh sau để tạo bảng jobs trong cơ sở dữ liệu:

```
php artisan queue:table
php artisan migrate
```

*Ghi chú: Khởi động sẵn ứng dụng bằng php artisan serve ở một cửa sổ Terminal khác.*

#### Bước 2. Tạo lớp Mailable hỗ trợ Queue:

Tạo một Class để quản lý nội dung bức thư. Chạy lệnh sau trong Terminal:

```
php artisan make:mail WelcomeRegisteredUserMail
```

Mở file app/Mail/WelcomeRegisteredUserMail.php vừa được sinh ra. **Điểm quan trọng nhất ở đây là bạn phải thêm giao diện (interface) ShouldQueue** để Laravel biết thư này cần chạy ngầm.

```
namespace App\Mail;
```

```
use App\Models\User;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue; // 1. BẮT BUỘC KHAI BÁO
use Illuminate\Mail\Mailable;
use Illuminate\Mail\Mailables\Content;
use Illuminate\Mail\Mailables\Envelope;
use Illuminate\Queue\SerializesModels;

// 2. Thêm "implements ShouldQueue" vào sau tên class
class WelcomeRegisteredUserMail extends Mailable implements ShouldQueue
{
    use Queueable, SerializesModels;

    // Khai báo biến public để truyền dữ liệu người dùng ra View
    public $user;

    /**
     * Hàm khởi tạo nhận vào đối tượng User vừa đăng ký
     */
    public function __construct(User $user)
    {
        $this->user = $user;
    }

    /**
     * Thiết lập Tiêu đề thư (Subject)
     */
    public function envelope(): Envelope
    {
        return new Envelope(
            subject: 'Chào mừng bạn gia nhập hệ thống của chúng tôi!',
        );
    }

    /**
     * Trỏ đến file giao diện HTML của thư
     */
    public function content(): Content
    {

```

```
        return new Content(  
            view: 'emails.auth.welcome', // Sẽ tạo file này ở Bước 3  
        );  
    }  
}
```

### Bước 3. Thiết kế giao diện HTML cho thư:

Tạo cấu trúc thư mục resources/views/emails/auth và tạo file welcome.blade.php bên trong đó.

Thêm đoạn mã HTML cơ bản sau (có thể tự thiết kế thêm CSS tùy ý):

```
        Chào mừng thành viên mới  
  
        Đăng ký tài khoản thành công!  
  
        Xin chào {{ $user->name }},  
  
        Cảm ơn bạn đã tin tưởng và đăng ký tài khoản trên hệ thống của chúng tôi.  
  
        Tài khoản của bạn đã được liên kết với email: {{ $user->email }}  
  
        Bạn có thể đăng nhập ngay bây giờ để trải nghiệm các tính năng  
        tuyệt vời nhất!  
  
        Đây là email gửi tự động thông qua Background Job. Vui lòng  
        không phản hồi.
```

### Bước 4. Kích hoạt gửi thư khi người dùng Đăng ký (Breeze Controller):

Chúng ta sẽ chèn lệnh gửi thư vào ngay sau khi hệ thống lưu tài khoản mới vào cơ sở dữ liệu.

Mở file app/Http/Controllers/Auth/RegisteredUserController.php (file này sinh ra từ Laravel Breeze ở Lab 10). Cập nhật hàm store như sau:

```
namespace App\Http\Controllers\Auth;  
  
use App\Http\Controllers\Controller;  
use App\Models\User;
```

## Lab 12 - Xây dựng tính năng tự động gửi email

```
use Illuminate\Auth\Events\Registered;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rules;
use Illuminate\View\View;

// BỎ SUNG 2 THƯ VIỆN NÀY ĐỂ GỬI MAIL
use Illuminate\Support\Facades\Mail;
use App\Mail>WelcomeRegisteredUserMail;

class RegisteredUserController extends Controller
{
    // ... (hàm create giữ nguyên) ...

    public function store(Request $request): RedirectResponse
    {
        $request->validate([
            'name' => ['required', 'string', 'max:255'],
            'email' => ['required', 'string', 'lowercase', 'email',
                'max:255', 'unique:'.User::class],
            'password' => ['required', 'confirmed',
                Rules\Password::defaults()],
        ]);

        // 1. Lưu User mới vào Database
        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
        ]);

        event(new Registered($user));

        // =====
        // 2. PHÁT LỆNH GỬI MAIL CHÀO MỪNG
        // Do WelcomeRegisteredUserMail có implements ShouldQueue,
        // hàm send() tự động chuyển thành queue(), đẩy vào DB chứ không
        // làm treo Web.
        // =====
        Mail::to($user->email)->send(new WelcomeRegisteredUserMail($user));
    }
}
```

```
// 3. Đăng nhập và chuyển hướng
Auth::login($user);

return redirect(route('dashboard', absolute: false));
}
```

### Bước 5. Kiểm thử và Kích hoạt Tiến trình nền (Worker):

Đây là bước quan trọng nhất để chứng minh Queue đang hoạt động. Hãy làm theo đúng trình tự để quan sát hiện tượng:

1. Đảm bảo bạn **CHƯA** chạy lệnh worker nào ở Terminal.
2. Mở trình duyệt, vào trang Đăng ký (Register) của ứng dụng (<http://localhost:8000/register>).
3. Điền thông tin đăng ký (Ví dụ: tên là MSSV của bạn) và bấm Đăng ký. Bạn sẽ thấy trang web load vào Dashboard **ngay lập tức**.
4. Mở CSDL, kiểm tra bảng jobs. Bạn sẽ thấy có 1 dòng dữ liệu xuất hiện (đó chính là bức thư đang nằm chờ).
5. Mở Mailpit (<http://localhost:8025>). Bạn sẽ thấy **chưa có** thư nào mới.
6. Mở một cửa sổ Terminal **mới**, gõ lệnh kích hoạt Worker:

```
php artisan queue:work
```

7. Ngay khi bấm Enter, Terminal sẽ báo Processing bức thư và sau đó báo Processed (Màu xanh).
8. Kiểm tra lại Mailpit, bức thư chào mừng đã xuất hiện! Dữ liệu trong bảng jobs cũng đã tự động bị xóa.

### BÀI TẬP MỞ RỘNG

Các kịch bản thực tế phức tạp hơn trong hệ thống Hàng đợi (Queue) và Mail của Laravel.

#### Bài tập mở rộng 1: Gửi Email trì hoãn (Delayed Jobs)

**Bối cảnh:** Trong thực tế, các chiến dịch Marketing thường không gửi email dồn dập ngay lập tức. Sau khi người dùng đăng ký, hệ thống gửi ngay email Chào mừng (đã

làm ở Lab 12). Tuy nhiên, hệ thống cần gửi thêm một email "Hướng dẫn sử dụng hệ thống" **sau đúng 10 phút** kể từ lúc đăng ký.

**Yêu cầu:**

1. Tạo thêm một lớp Mailable mới tên là UserGuideMail (nhớ implements ShouldQueue).
2. Sửa logic trong Controller đăng ký: Đẩy UserGuideMail vào hàng đợi nhưng thiết lập **độ trễ (delay) là 10 phút** trước khi Worker được phép xử lý.

**Gợi ý:** Tìm hiểu phương thức delay() khi gọi Mailable trong Laravel.

```
Mail::to($user->email)
    ->later(now()->addMinutes(10), new UserGuideMail($user));
```

## Bài tập mở rộng 2: Gửi Email kèm File đính kèm (Attachments)

**Bối cảnh:** Hệ thống yêu cầu khi người dùng đăng ký thành công, email chào mừng gửi tới phải đính kèm một file PDF chứa "Điều khoản dịch vụ và Chính sách bảo mật" để người dùng lưu trữ.

**Yêu cầu:**

1. Tạo một file PDF giả định (ví dụ: chinh-sach.pdf) và đặt vào thư mục public hoặc storage của project.
2. Cập nhật file WelcomeRegisteredUserMail.php để tự động đính kèm file PDF này vào bức thư khi gửi đi.

**Gợi ý:** Sử dụng hàm attachments() bên trong file cấu hình thư của Laravel 11/12.

```
public function attachments(): array
{
    return [
        Attachment::fromPath(public_path('chinh-sach.pdf')),
    ];
}
```

## Bài tập mở rộng 3: Gửi thông báo cho Admin (Multi-Jobs)

**Bối cảnh:** Để kiểm soát lượng người dùng mới, Ban quản trị (Admin) yêu cầu hệ thống phải gửi một email thông báo ngắn gọn tới địa chỉ admin@mywebsite.com mỗi khi có một tài khoản khách hàng mới vừa đăng ký.

**Yêu cầu:**

1. Khi người dùng bấm Đăng ký, hệ thống sẽ đẩy **2 tác vụ (Jobs)** vào hàng đợi cùng lúc:
  - Tác vụ 1: Gửi email chào mừng cho người dùng (Đã làm).
  - Tác vụ 2: Gửi email thông báo "Có User mới đăng ký" tới địa chỉ email của Admin.
2. Minh chứng được bảng jobs trong Database hiển thị 2 records tồn đọng trước khi bật Worker.

**Bài tập mở rộng 4: Mô phỏng lỗi và Cấu hình bảng Failed Jobs**

**Bối cảnh:** Điều gì xảy ra nếu Worker đang cố gắng gửi email nhưng mạng bị đứt, hoặc cấu hình SMTP bị sai mật khẩu? Mặc định, Worker sẽ báo lỗi và tác vụ đó có thể bị mất. Laravel cung cấp cơ chế lưu trữ các tác vụ thất bại vào bảng failed\_jobs để Admin có thể xem xét và gửi lại (Retry) sau khi sửa lỗi mạng.

**Yêu cầu:**

1. Chạy lệnh tạo bảng lưu trữ tác vụ lỗi: `php artisan queue:failed-table` và `php artisan migrate`.
2. Cố tình nhập sai cấu hình cổng SMTP của Mailpit trong file `.env` (ví dụ đổi port thành 9999).
3. Đăng ký tài khoản mới và chạy Worker.
4. Chụp màn hình Terminal báo lỗi (Màu đỏ) và chụp bảng failed\_jobs trong Database lưu trữ tác vụ gửi mail bị thất bại này.
5. Sửa lại file `.env` cho đúng, sau đó dùng lệnh Terminal để **Retry (Thử chạy lại)** tác vụ vừa thất bại mà không cần phải nhờ người dùng đăng ký lại.

**BIỂU MẪU BÁO CÁO NỘP BÀI LAB 12**

*Sinh viên copy toàn bộ nội dung phần này vào file Word `MSSV_HoTen_Lab12.docx`, điền đầy đủ minh chứng để nộp bài.*

**LAB 12: GỬI EMAIL CHÀO MỪNG QUA QUEUE**

**Thông tin sinh viên:**

– Họ và tên: .....

- Mã số sinh viên (MSSV): .....

### 1. Minh chứng Code:

- Yêu cầu dán nội dung file WelcomeRegisteredUserMail.php (Đảm bảo có từ khóa implements ShouldQueue).

*[Sinh viên dán code dạng text vào đây]*

### 2. Minh chứng Queue hoạt động (Chưa bật Worker):

- **Hành động:** Đăng ký một tài khoản mới có tên là MSSV của bạn (vd: Nguyễn Văn A - 2110xxxx). Tuyệt đối **KHÔNG** bật lệnh queue:work.
- **Hình 1:** Chụp toàn màn hình phần mềm quản lý Database (phpMyAdmin / TablePlus) hiển thị bảng jobs đang có 1 record dữ liệu tác vụ tồn đọng.

*[Sinh viên chèn ảnh 1 vào đây]*

### 3. Minh chứng Worker xử lý thành công:

- **Hành động:** Mở Terminal chạy lệnh php artisan queue:work.
- **Hình 2:** Chụp 1 ảnh gồm 2 cửa sổ: Một nửa là cửa sổ Terminal báo chữ Processed màu xanh, nửa kia là giao diện **Mailpit** hiển thị rõ bức thư chào mừng gửi tới tên tài khoản chứa MSSV của bạn.

*[Sinh viên chèn ảnh 2 vào đây]*